

VIET NAM NATIONAL UNIVERSITY, HCMC

UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

ASSIGNMENT 1

CS419 - Introduction to Information Retrieval

Student:

Tran Nguyen Phuc Thinh

Student ID: 23125072

Lecturer:

Le Trung Nghia

Nguyen Vinh Tiep

Teaching Assistant:

Do Trong Le

Huynh Viet Tham

Nguyen Trong Thuan

December 9, 2025



Contents

1	Discovering the website structure	2
1.1	Inspecting HTML Elements	2
1.2	Hidden API Endpoints	2
2	Crawling Strategy and Implementation	3
2.1	Timeline (pages) Problem	3
2.2	Skipping Unnecessary Images	3
2.3	Implementation	3
2.3.1	Libraries	3
2.3.2	Main flow	4
2.3.3	Error Handling and Robustness	4
2.3.4	Dataset Storage Design	5
	References	6

1 Discovering the website structure

1.1 Inspecting HTML Elements

The first step of the assignment was understanding how the Tuoi Tre Online website structures its content. Using Opera Developer Tools, I examined the HTML for multiple articles and identified the core elements necessary for crawling:

- `<h1>`: article title
- `<meta property="article:published_time">`: publish date
- `<meta property="article:section">`: category
- `<p>`: tags inside main content area
- `<img>`: containing image paths

This allowed me to extract structured metadata consistently.

One important finding was that the audio player is not exposed via a normal `<audio>` tag. Instead of embedding the actual .m4a file, the site uses a JavaScript-based player that only fetches the audio file when the play button is pressed. Because of this, I could not extract audio URLs directly from the HTML. This led to the deeper investigation described next.

1.2 Hidden API Endpoints

Through inspecting the network tab, I discover these hidden API endpoints for fetching data:

- **getlist-comment.api**: to get the list of comments and their reactions for a specific post. The URL comes with an API key that I can use to fetch comments from a post. There are many other parameters to pass into the url, but the most important one is the post ID.
- **getcount-comment.api**: found in the list of APIs but I do not see anywhere to pass in the parameters to fetch the number of comments, so I use getlist-comment.api to count the number of comments to found posts with over 20 comments.
- **showvote-reaction.htm**: fetch reactions for a post.
- **Text-to-speech audio endpoint**: not inside HTML structure, only discoverable after I have pressed the play button in an audio. Every post have an AI-generated audio, with the url having the format:

`https://tts.mediacd.vn/YYYY/MM/DD/tuoitre-nu-1-POSTID.m4a`

I fetch all audio with the default configuration **nu-1**, which is Southern female voice.

2 Crawling Strategy and Implementation

2.1 Timeline (pages) Problem

In a category, when I surf deep down in the page, the address will change its page accordingly, for example **thoi-su/trang-1.htm**, **thoi-su/trang-2.htm**,... However, when I try typing these addresses to the search bar, they almost always shows the same content. Therefore, these page indexing are very inconsistent and cannot be used for crawling new posts.

After further inspection, I found out that the actual pagination uses the url of format:

`https://tuoitre.vn/timeline/3/trang-2.htm`

With the number 3 likely to be the id for the category (which is thoi-su in this case). The page of this thoi-su category contains other categories as well, but many of the posts are still of the thoi-su category, so this is not a big concern. The problem of pagination is solved.

However, now we need to find the actual mapping from the categories to id to put into the url. After investigation, I found that in comments fetched from posts with comments, there is an attribute **zone_id** which is exactly the category id for the timeline. This is true for all the categories. Using this, I first write a function to found a post with at least one comment and belong to a category to find the id for that category. Then, I save the mapping in a .json file.

2.2 Skipping Unnecessary Images

During image fetching, there are some images that are just website assets and not relevant to the post specifically, which includes

- **Avatar.jpg**
- **banner_introduce_tto.png**
- **banner_gg_news_v2.png**
- **logotuoitrepro-02.svg**
- **logottonew.svg**
- **userdeffault.jpg**
- **newslette.png**
- **author_default.png**.

So I skip these during fetching to keep it clean.

2.3 Implementation

2.3.1 Libraries

I use the Python libraries **requests**, **BeautifulSoup**, for crawling data, along with other helper libraries.

2.3.2 Main flow

1. Load categories and build timeline mapping
2. For each category:
 - Crawl timeline pages using `/timeline/<zone>/trang-N.htm`
 - Extract post urls
 - Filter urls belonging to the category
3. For each article:
 - Parse metadata
 - Build TTS audio URL, download **.m4a** file
 - Filter urls belonging to the category
 - Download images (filtered)
 - Fetch reactions
 - Fetch comments
 - Save all data

For the requirement "at least one post has more than 20 comments", I use a variable to log if I have crawled a post with more than 20 comments. If I have reached the last post to crawl in the last category, and have not found any page with 20 comments, I continue to find in pages in that category until I find one.

2.3.3 Error Handling and Robustness

The crawler is designed to fail gracefully and continue running in the presence of network issues or malformed data:

- `safe_get()`: all HTTP requests go through a wrapper that checks `robots.txt`, sets timeouts, enforces a delay between requests, catches exceptions, logs warnings, and returns `None` on failure instead of raising errors.
- Defensive JSON parsing: calls to the comment and reaction APIs wrap `resp.json()` and `json.loads()` in `try/except`. Invalid or unexpected responses are logged and skipped, rather than crashing the crawler.
- Tolerant data access: comment and reaction objects are treated as partially missing or inconsistent. The code checks for keys such as `"id"` and `"zone_id"`, skips non-dict entries, and stops paginating when empty pages are returned.
- Safe media and mapping handling: missing timeline IDs or failed category pages result in logged warnings and the category being skipped. Media downloads check for existing files and create directories with `exist_ok=True`.
- Logging: all key steps (requests, mapping discovery, comment fetching, media downloads) are recorded with timestamps and levels, making it easy to trace errors post hoc without interrupting the crawl.

2.3.4 Dataset Storage Design

Each article is stored as a JSON file in:

`data/<post_id>.json`

The file contains the following fields:

- **postId**: the unique identifier of the article, extracted from the URL or page metadata.
- **title**: the article's headline as displayed on the page.
- **content**: the full textual body of the article, with paragraph breaks preserved.
- **author**: the name of the article's author.
- **date**: the publication timestamp in ISO 8601 format (e.g., 2025-12-06T20:17:52+07:00).
- **category**: the article's actual category.
- **url**: the canonical URL of the article.
- **audio_podcast**: the local path to the downloaded audio file or `null` if unavailable.
- **images**: list of local filesystem paths to downloaded article-specific images, excluding global assets such as logos or banners.
- **vote_reactions**: dictionary containing reaction statistics returned by the reaction API.
- **comments**: hierarchical list of top-level comments and their replies, each including text, author, date, and reaction metadata.
- **total_comment_count**: total number of raw comment objects retrieved from all comment pages.

Images are stored under:

`images/<post_id>/`

And audio files are saved to:

`audio/<post_id>.m4a`

This structured layout is scalable, modular, and suitable for downstream data analysis tasks such as text mining, sentiment analysis, or information retrieval.

References

- [1] GeeksforGeeks. *Implementing Web Scraping in Python with BeautifulSoup*. 26 **july** 2025.
URL: <https://www.geeksforgeeks.org/python/implementing-web-scraping-python-beautiful-soup/> (**urlseen** 09/12/2025).